

0.80.2. x et y rangés ensemble : struct — une fonction dans un fichier

Le 0.80.1, en mieux : l'intérieur de la boucle devient une fonction, `tour_de_jeu()`, rangée dans `jeu.h`. La boucle n'a plus qu'une ligne.

1. Ce qu'il faut comprendre

C'est le 0.80.1, en mieux : au lieu de verser des lignes en vrac au milieu de la boucle, on les range dans une **fonction**, `tour_de_jeu()`, dans le fichier `jeu.h`.

Ce qui est nouveau ici : `void`. Au 0.33, notre fonction **rendait** une valeur (`uint8_t`, puis `return colonne;`). Celle-ci ne rend rien : elle **fait** quelque chose. On l'écrit `void tour_de_jeu() { ... }` : `void` veut dire « rien », et les parenthèses vides veulent dire qu'elle ne reçoit rien.

Le `#include "jeu.h"` revient tout en haut, avant `main`, comme au 0.67 : il apporte la **définition** de la fonction. La boucle, elle, se contente de l'**appeler** : `tour_de_jeu();`, une ligne.

Pourquoi c'est mieux : en ouvrant `jeu.h`, on voit une fonction complète, avec son nom et ses accolades : on sait ce qu'elle est. Et `main` se lit comme une phrase : « tant que le jeu tourne, fais un tour de jeu ».

L'ordre compte : `joueur` est déclaré **avant** le `#include "jeu.h"`, parce que la fonction s'en sert. Une variable doit exister avant qu'on écrive quelque chose qui l'utilise.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1779 octets** de cartouche.

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// Le nouveau type : la colonne et la ligne sous un seul nom (le 0.80).
```

2. UN ENREGISTREMENT — POSITION

```
struct Position { uint8_t x, y; };
```

3. UNE VARIABLE — JOUEUR

```
Position joueur;          // joueur.x et joueur.y

// La fonction tour_de_jeu : voir l'onglet jeu.h. Elle se sert de joueur,
// déclaré juste au-dessus : l'ordre compte.
#include "jeu.h"
```

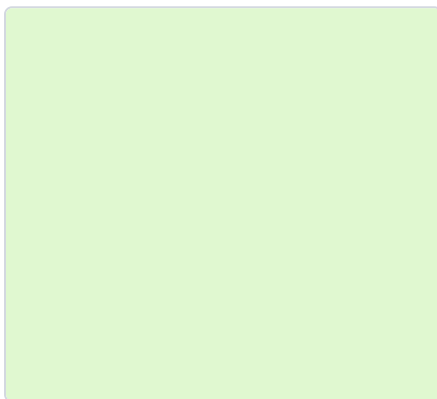
4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    joueur.x = 10;    // départ : (10, 0)
    joueur.y = 0;

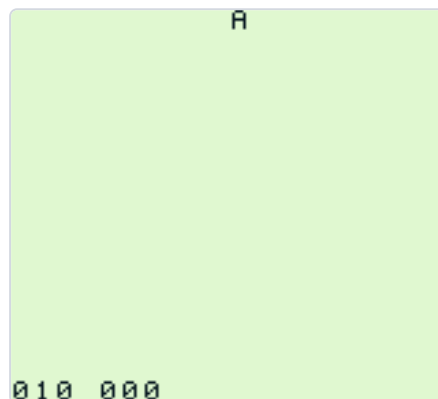
    while (true) {
        tour_de_jeu(); // le changement : UNE ligne, l'appel de la fonction
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Exactement le 0.80 : un A qui suit la croix. La boucle ne contient plus qu'un appel : `tour_de_jeu()`.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A part de (10, 0)
- ✓ la croix le déplace, depuis l'autre fichier (6, 2)
- ✓ il est à la place que disent `joueur.x` et `joueur.y`

Ensuite : **0.81. Ramasser une pièce : le P** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++